

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №6

по дисциплине: "Логика и основы алгоритмизации в инженерных задачах"

на тему: "Унарные бинарные операции над графами"

Выполнил:

Студент группы 24ВВВ3

Макунькин А.М.

Мелюшев М.М.

Принял:

к.н., доцент, Юрова О. В.

к.т.н., Деев М.В.

Пенза 2025

Цель

Изучить унарные и бинарные операции над графами

Теоретическая часть

Все унарные операции над графами можно объединить в две группы. Первую группу составляют операции, с помощью которых из исходного графа G_1 , можно построить граф G_2 с меньшим числом элементов. В группу входят операции удаления ребра или вершины, отождествления вершин, стягивание ребра. Вторую группу составляют операции, позволяющие строить графы с большим числом элементов. В группу входят операции расщепления вершин, добавления ребра.

Отождествление вершин. В графе G_1 выделяются вершины u, v . Определяют окружение Q_1 вершины u , и окружение Q_2 вершины v , вычисляют их объединение $Q = Q_1 \cup Q_2$. Затем над графом G_1 выполняются следующие преобразования:

- из графа G_1 удаляют вершины u, v ($H_1 = G_1 - u - v$);
- к графу H_1 присоединяют новую вершину z ($H_1 = H_1 + z$);
- вершину z соединяют ребром с каждой из вершин $w_i \in Q$ ($G_2 = H_1 + zw_i, i = 1, 2, 3, \dots$).

Стягивание ребра. Данная операция является операцией отождествления смежных вершин u, v в графе G_1 .

Наиболее важными бинарными операциями являются: объединение, пересечение, декартово произведение и кольцевая сумма.

Объединение. Граф G называется объединением или наложением графов G_1 и G_2 , если $V_G = V_1 \cup V_2$; $E_G = E_1 \cup E_2$ (рис. 1).

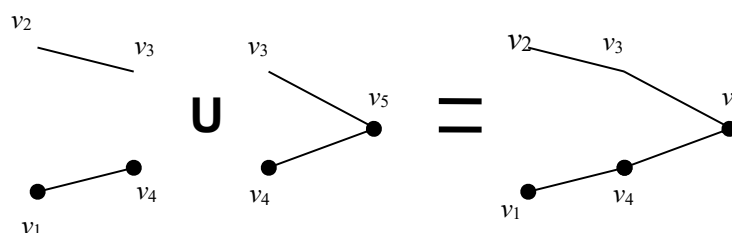


Рис. 1. Объединение графов G_1, G_2

Объединение графов G_1 и G_2 называется дизъюнктивным, если $V_1V_2 = \emptyset$. При дизъюнктивном объединении никакие два из объединяемых графов не должны иметь общих вершин.

Пересечение. Граф G называется пересечением графов G_1, G_2 , если $V_G = V_1V_2$ и $U_G = U_1U_2$ (рис.2). Операция "пересечения" записывается следующим образом: $G = G_1G_2$.

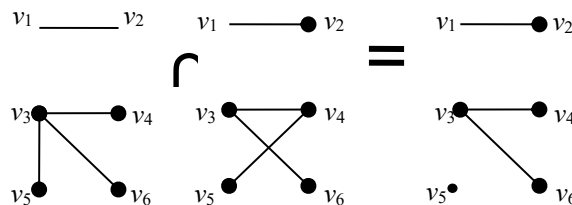


Рис.2. Пересечение графов G_1, G_2 .

Декартово произведение. Граф G называется декартовым произведением графов G_1 и G_2 если $V_G = V_1 \times V_2$ —декартово произведение множеств вершин графов G_1, G_2 , а множество ребер U_G задается следующим образом: вершины (z_i, v_k) и (z_j, v_l) смежны в графе G тогда и только тогда, когда $z_i = z_j (i = j)$, а v_k и v_l смежны в G_2 или $v_k = v_l (k = l)$, смежны в графе G_1 (см. рис.3).

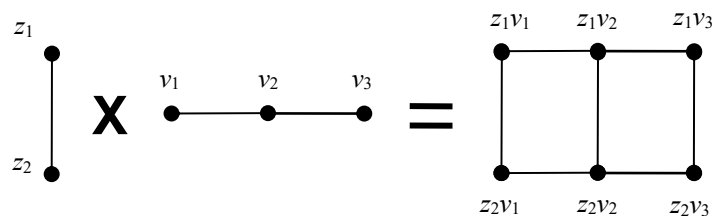


Рис. 3. Декартово произведение графов G_1, G_2

Кольцевая сумма графов представляет граф, который не имеет изолированных вершин и состоит из ребер, присутствующих либо в первом исходном графе, либо во втором. Кольцевая сумма определяется следующим соотношением: $G = G_1 \oplus G_2$ (рис.4).

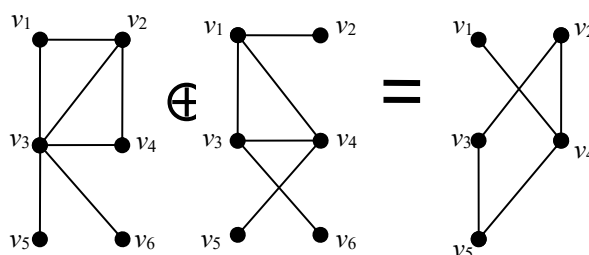


Рис.4. Кольцевая сумма графов G_1, G_2

Лабораторное задание:

1. Сгенерируйте (используя генератор случайных чисел) две матрицы M_1, M_2 смежности неориентированных помеченных графов G_1, G_2 . Выведите сгенерированные матрицы на экран.

2. Для матричной формы представления графов выполните операцию:

- а) отождествления вершин
- б) стягивания ребра
- в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры.

Результат выполнения операции выведите на экран.

3. Для матричной формы представления графов выполните операцию:

- а) объединения $G = G_1 \cup G_2$
- б) пересечения $G = G_1 \cap G_2$
- в) кольцевой суммы $G = G_1 \oplus G_2$

Результат выполнения операции выведите на экран.

Результат программы

```
Введите количество вершин графа: 4
=== ЗАДАНИЕ 1 ===
Матрица смежности G1:
1 1 1 0
1 0 1 0
1 1 0 1
0 0 1 1

Матрица смежности G2:
1 1 1 1
1 0 1 1
1 1 1 1
1 1 1 0
```

Рисунок 1 – генерация матриц смежности

```

=== ЗАДАНИЕ 2 ===

--- ОТОЖДЕСТВЛЕНИЕ ВЕРШИН ---
Для графа G1 – введите две вершины для отождествления (1-4): 1 2
Результат отождествления вершин 1 и 2 в G1:
0 1 1
1 1 0
1 0 1

Для графа G2 – введите две вершины для отождествления (1-4): 1 2
Результат отождествления вершин 1 и 2 в G2:
1 1 1
1 0 1
1 1 1

```

Рисунок 2 – Отождествление вершин

```

--- СТЫГИВАНИЕ РЕБРА ---
Для графа G1 – введите две вершины для стягивания ребра (1-4): 1 2
Результат стягивания ребра между 1 и 2 в G1:
0 1 1
1 1 0
1 0 1

Для графа G2 – введите две вершины для стягивания ребра (1-4): 1 2
Результат стягивания ребра между 1 и 2 в G2:
1 1 1
1 0 1
1 1 1

```

Рисунок 3 – Стягивание ребра

```

--- РАСЩЕПЛЕНИЕ ВЕРШИНЫ ---
Для графа G1 – введите вершину для расщепления (1-4): 1 2
Результат расщепления вершины 1 в G1:
1 1 1 0 1
1 0 1 0 1
1 1 0 1 1
0 0 1 1 0
1 1 1 0 1

Для графа G2 – введите вершину для расщепления (1-4): Результат расщепления вершины 2 в G2:
1 1 1 1 1
1 0 1 1 1
1 1 1 1 1
1 1 1 0 1
1 1 1 1 0

```

Рисунок 4 – Расщепление вершины

```
=== ЗАДАНИЕ 3 ===  
Объединение G1 и G2:  
1 1 1 1  
1 0 1 1  
1 1 1 1  
1 1 1 1  
  
Пересечение G1 и G2:  
1 1 1 0  
1 0 1 0  
1 1 0 1  
0 0 1 0  
  
Кольцевая сумма G1 и G2:  
0 0 0 1  
0 0 0 1  
0 0 1 0  
1 1 0 1
```

Рисунок 5 – Объединение, пересечение и кольцевая сумма

Листинг

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

int** generate_adjacency_matrix(int n) {
    int** matrix = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        matrix[i] = (int*)malloc(n * sizeof(int));
        for (int j = 0; j < n; j++) {
            if (i == j) {
                matrix[i][j] = rand() % 2;
            }
            else if (i < j) {
                matrix[i][j] = rand() % 2;
            }
            else {
                matrix[i][j] = matrix[j][i];
            }
        }
    }
    return matrix;
}

void print_matrix(int** matrix, int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
    printf("\n");
}

void free_matrix(int** matrix, int n) {
    for (int i = 0; i < n; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

int** identify_vertices(int** adj, int n, int u, int v) {
    int new_n = n - 1;
    int** new_adj = (int**)malloc(new_n * sizeof(int*));
    for (int i = 0; i < new_n; i++) {
        new_adj[i] = (int*)calloc(new_n, sizeof(int));
    }

    int z = new_n - 1;

    for (int i = 0, new_i = 0; i < n; i++) {
        if (i == u || i == v) continue;

        for (int j = 0, new_j = 0; j < n; j++) {
            if (j == u || j == v) continue;

            new_adj[new_i][new_j] = adj[i][j];
            new_j++;
        }
        new_i++;
    }

    for (int i = 0, new_i = 0; i < n; i++) {
        if (i == u || i == v) continue;

        if (adj[i][u] == 1 || adj[i][v] == 1) {
            new_adj[new_i][z] = 1;
        }
    }
}
```

```

        new_adj[z][new_i] = 1;
    }
    new_i++;
}

if (adj[u][u] == 1 || adj[v][v] == 1 || adj[u][v] == 1) {
    new_adj[z][z] = 1;
}

return new_adj;
}

int** contract_edge(int** adj, int n, int u, int v) {
    if (adj[u][v] == 0) {
        printf("Ребро между вершинами отсутствует.\n");
        return adj;
    }
    return identify_vertices(adj, n, u, v);
}

int** split_vertex(int** adj, int n, int u) {
    int new_n = n + 1;
    int** new_adj = (int**)malloc(new_n * sizeof(int*));
    for (int i = 0; i < new_n; i++) {
        new_adj[i] = (int*)calloc(new_n, sizeof(int));
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            new_adj[i][j] = adj[i][j];
        }
    }

    int new_vertex = new_n - 1;

    new_adj[u][new_vertex] = 1;
    new_adj[new_vertex][u] = 1;

    for (int i = 0; i < n; i++) {
        if (adj[u][i] == 1) {
            if (i == u) {
                new_adj[new_vertex][new_vertex] = 1;
            }
            else {
                new_adj[new_vertex][i] = 1;
                new_adj[i][new_vertex] = 1;
            }
        }
    }

    return new_adj;
}

int** union_graphs(int** adj1, int** adj2, int n) {
    int** result = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        result[i] = (int*)malloc(n * sizeof(int));
        for (int j = 0; j < n; j++) {
            result[i][j] = (adj1[i][j] == 1 || adj2[i][j] == 1) ? 1 : 0;
        }
    }
    return result;
}

int** intersect_graphs(int** adj1, int** adj2, int n) {
    int** result = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        result[i] = (int*)malloc(n * sizeof(int));
        for (int j = 0; j < n; j++) {
            result[i][j] = (adj1[i][j] == 1 && adj2[i][j] == 1) ? 1 : 0;
        }
    }
}

```

```

    }
}
return result;
}

int** ring_sum_graphs(int** adj1, int** adj2, int n) {
    int** result = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        result[i] = (int*)malloc(n * sizeof(int));
        for (int j = 0; j < n; j++) {
            result[i][j] = (adj1[i][j] != adj2[i][j]) ? 1 : 0;
        }
    }
    return result;
}

int main() {
    setlocale(LC_ALL, "Russian");
    srand(time(NULL));

    int n;
    printf("Введите количество вершин графа: ");
    scanf("%d", &n);

    printf("=== ЗАДАНИЕ 1 ===\n");
    int** adj1 = generate_adjacency_matrix(n);
    int** adj2 = generate_adjacency_matrix(n);

    printf("Матрица смежности G1:\n");
    print_matrix(adj1, n);

    printf("Матрица смежности G2:\n");
    print_matrix(adj2, n);

    printf("=== ЗАДАНИЕ 2 ===\n");

    printf("\n--- ОТОЖДЕСТВЛЕНИЕ ВЕРШИН ---\n");

    printf("Для графа G1 - введите две вершины для отождествления (1-%d): ", n);
    int u1, v1;
    scanf("%d %d", &u1, &v1);
    int** result_identify1 = identify_vertices(adj1, n, u1 - 1, v1 - 1);
    printf("Результат отождествления вершин %d и %d в G1:\n", u1, v1);
    print_matrix(result_identify1, n - 1);

    printf("Для графа G2 - введите две вершины для отождествления (1-%d): ", n);
    int u2, v2;
    scanf("%d %d", &u2, &v2);
    int** result_identify2 = identify_vertices(adj2, n, u2 - 1, v2 - 1);
    printf("Результат отождествления вершин %d и %d в G2:\n", u2, v2);
    print_matrix(result_identify2, n - 1);

    printf("\n--- СЯГИВАНИЕ РЕБРА ---\n");

    printf("Для графа G1 - введите две вершины для стягивания ребра (1-%d): ", n);
    scanf("%d %d", &u1, &v1);
    int** result_contract1 = contract_edge(adj1, n, u1 - 1, v1 - 1);
    printf("Результат стягивания ребра между %d и %d в G1:\n", u1, v1);
    if (result_contract1 != adj1) {
        print_matrix(result_contract1, n - 1);
    }

    printf("Для графа G2 - введите две вершины для стягивания ребра (1-%d): ", n);
    scanf("%d %d", &u2, &v2);
    int** result_contract2 = contract_edge(adj2, n, u2 - 1, v2 - 1);
    printf("Результат стягивания ребра между %d и %d в G2:\n", u2, v2);
    if (result_contract2 != adj2) {
        print_matrix(result_contract2, n - 1);
    }
}

```

```

printf("\n--- РАСЩЕПЛЕНИЕ ВЕРШИНЫ ---\n");

printf("Для графа G1 - введите вершину для расщепления (1-%d): ", n);
scanf("%d", &u1);
int** result_split1 = split_vertex(adj1, n, u1 - 1);
printf("Результат расщепления вершины %d в G1:\n", u1);
print_matrix(result_split1, n + 1);

printf("Для графа G2 - введите вершину для расщепления (1-%d): ", n);
scanf("%d", &u2);
int** result_split2 = split_vertex(adj2, n, u2 - 1);
printf("Результат расщепления вершины %d в G2:\n", u2);
print_matrix(result_split2, n + 1);

printf("=== ЗАДАНИЕ 3 ===\n");

int** result_union = union_graphs(adj1, adj2, n);
printf("Объединение G1 и G2:\n");
print_matrix(result_union, n);

int** result_intersect = intersect_graphs(adj1, adj2, n);
printf("Пересечение G1 и G2:\n");
print_matrix(result_intersect, n);

int** result_ring_sum = ring_sum_graphs(adj1, adj2, n);
printf("Кольцевая сумма G1 и G2:\n");
print_matrix(result_ring_sum, n);

free_matrix(adj1, n);
free_matrix(adj2, n);
free_matrix(result_identify1, n - 1);
free_matrix(result_identify2, n - 1);
if (result_contract1 != adj1) {
    free_matrix(result_contract1, n - 1);
}
if (result_contract2 != adj2) {
    free_matrix(result_contract2, n - 1);
}
free_matrix(result_split1, n + 1);
free_matrix(result_split2, n + 1);
free_matrix(result_union, n);
free_matrix(result_intersect, n);
free_matrix(result_ring_sum, n);

return 0;
}

```

Вывод

В ходе выполнения лабораторной работы была разработана программа для выполнения заданий Лабораторной работы №6. В процессе выполнения работы были использованы знания о унарных и бинарных операций над графами.

Ссылка на удаленный репозиторий GitHub:

<https://github.com/LareklAREK/LiOAiIZ-2OURSE-.git>

<https://github.com/Motorrr/LiOAvIZ.git>